

IndiaLock Connect

Technical Design & Build Document — POC Build Guide (for use with Claude Code)

Document Version: 1.0

Companion To: IndiaLock Connect — Business Requirements Document v1.0

Protocol Source: TT808ELOCK Protocol Specification, Rev. A13 (2018-05-06)

Audience: Developers, and non-technical team members supervising the build

Date: September 2026

1. Purpose of This Document & How To Use It

This document is written so that it can be handed directly to an AI coding assistant (Claude Code) as a build specification, and so that a non-technical team member can still follow along and understand what is being built and why.

How to use this with Claude Code:

Give Claude Code this entire document as context, then ask it to build the POC section by section, following the 'Build Order' in Section 9. Each section is self-contained and can be handed over as its own task if you prefer to build incrementally (e.g., 'Build me the Protocol Parser module described in Section 5' or 'Build me the database schema in Section 7').

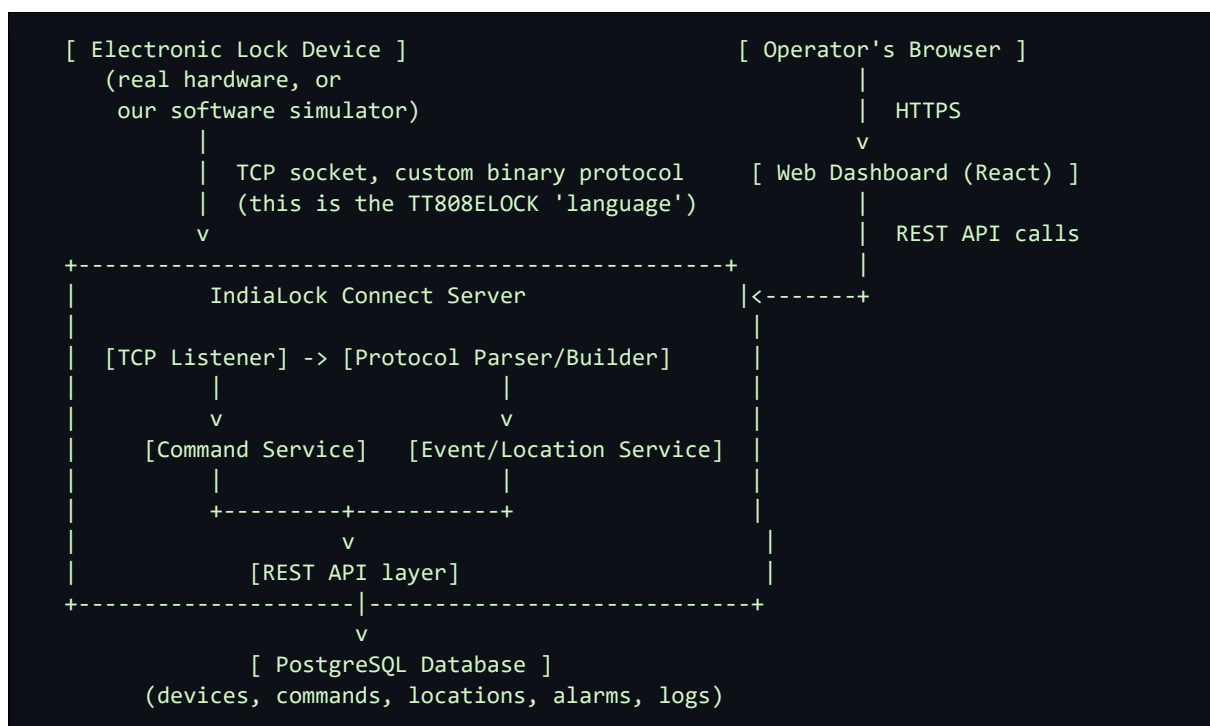
Every technical decision below is deliberately kept simple and explained in plain language first, followed by the precise technical detail a developer or AI tool needs.

2. System Overview

In plain terms: we are building a small web application with three moving parts.

1. A 'listener' program that keeps a phone-line-like connection open with the lock device, and understands the special language (protocol) it speaks.
2. A database that remembers everything that happens — every lock, every unlock, every location update, every alarm.
3. A web page (dashboard) where a human can see the lock's status and press buttons to lock/unlock it.

2.1 Architecture Diagram (described)



2.2 Two Ways the POC Can Run

Mode	What's connected	When to use it
Simulated mode	A small script pretends to be the lock device and connects to our server, playing back realistic scripted behaviour.	Immediately, before hardware arrives. Lets us build and test 90% of the platform.
Hardware mode	The real physical lock sample, with its own SIM card, connects to our server over the internet.	Once the vendor delivers the physical sample. No platform code changes needed — just point the real device at the same server.

3. What We Already Have vs. What We Need

3.1 What the Protocol Document Gives Us (Confirmed)

- The exact byte-level structure of every message (how to build a message and how to read one back).
- The exact commands for Seal, Unseal, Clear Alarm, Set Dynamic Password, and Change Local Password.
- The exact response/result codes the lock will send back (success, duplicate seal, low voltage, tamper, etc.).
- The registration, authentication, and heartbeat (keep-alive) handshake sequence.
- The GPS location report format, including speed, direction, and a set of alarm flags.
- A worked byte-level example for the message-escaping rule (Section 4.4.2 of the protocol doc) — useful as a test case.

3.2 What We Still Need (and where it comes from)

What we need	Why	Source
A physical lock sample with active SIM	To validate our software actually works with real hardware, not just on paper	Vendor delivery (already arranged)
The lock's default LockID, IP/port it will try to connect to, and any pre-set password	The device needs to be told (or already knows) where our server is	Ask vendor / check device configuration on arrival
A public IP address and open TCP port for our server	The lock connects over the mobile internet, so our server must be reachable from outside	Our own cloud/infra team
Confirmation of which protocol variant is on the actual hardware (7B lock vs 4G lock — protocol mentions both)	Some fields differ between lock hardware variants (e.g., 7-byte vs 10-byte cell tower info)	Check the physical unit / ask vendor once delivered
Any firmware-specific quirks not captured in the document	Real devices sometimes deviate slightly from written specs	Discovered during hardware testing; escalate to vendor if found
Key principle for the POC: We do not wait on any of the 'What We Still Need' items to start building. We build and test the entire platform using a software simulator first (Section 6.4), so the moment a physical lock arrives, we simply point it at our server and validate.		

4. Protocol Reference Summary (Plain-Language + Precise)

This section translates the relevant parts of the TT808ELOCK protocol document into exact rules our code must follow. Every number below (byte positions, hex codes) is taken directly from the supplied protocol document.

4.1 Message Structure (Every Message Looks Like This)

Every message sent between the lock and our server is wrapped the same way, like an envelope:

Part	Plain meaning	Technical detail
Start marker	Marks 'a message begins here'	Byte 0x7E
Message Header	Says which command this is, how long the message is, and which device it's from	Message ID (2 bytes) + Body Attributes (2 bytes) + Terminal ID (6 bytes, BCD) + Sequence Number (2 bytes)
Message Body	The actual content — e.g., 'seal this lock' or 'here is my GPS location'	Varies per message type — see Section 5
Checksum	A single byte used to detect if the message got corrupted in transit	XOR of every byte from the start of the header through the end of the body
End marker	Marks 'the message ends here'	Byte 0x7E

4.2 Byte-Escaping Rule (Important — implement exactly)

Because 0x7E is used as the 'start/end' marker, if that exact byte value happens to appear inside the real data, it must be disguised so it isn't mistaken for the start/end of a message. This is called escaping. The rule:

- Replace any 0x7E found inside the header/body/checksum with the two bytes: 0x7D 0x02
- Replace any 0x7D found inside the header/body/checksum with the two bytes: 0x7D 0x01

When reading a message, do this in reverse before checking the checksum.

Worked Example (from the protocol document — use this as a test case)

```
Raw data to send:      0x30 0x7E 0x08 0x7D 0x55
After escaping: 0x7E 0x30 0x7D 0x02 0x08 0x7D 0x01 0x55 0x7E
                  (start) ..... escaped bytes ..... (end)
```

4.3 Checksum Rule

The checksum is calculated by taking the first byte after the start marker, and XOR-ing it with every following byte, all the way up to (but not including) the checksum byte itself. XOR is a simple bit-comparison operation available in every programming language.

4.4 Message Header Field Layout

Byte Position	Field	Size & Type	Meaning
0	Message ID	2 bytes (WORD)	Identifies the type of message (e.g., 0x0100 = device registration).
2	Body Attributes	2 bytes (WORD)	Encodes the length of the message body, plus flags for encryption and multi-part messages. For the POC, we treat this as: bottom 10 bits = body length, rest = 0 (no encryption, no multi-part).
4	Terminal ID	6 bytes (BCD)	The unique ID of the lock device — like a phone number for the lock.
10	Sequence Number	2 bytes (WORD)	A running counter starting at 0, incremented with every message sent, used to match requests with responses.

4.5 Data Types Used (Plain Explanation)

Type	Plain meaning
BYTE	A single number from 0–255 (one raw byte of data).
WORD	A number from 0–65,535, stored using 2 bytes. Bigger-value-first ('big-endian') byte order.
DWORD	A larger number, stored using 4 bytes. Also big-endian.
BCD	A way of storing digits where each half-byte represents one decimal digit (e.g., used for dates and IDs).
STRING	Text, encoded in GBK (a Chinese-compatible text encoding) — for the POC, treat as UTF-8/ASCII-safe text and note GBK conversion as a follow-up if non-English text is ever needed.

5. Message Catalog — Only What the POC Needs

The full protocol document defines dozens of message types (photos, video, CAN-bus data, IC cards, etc.). The POC only needs the messages below. Build support for exactly these — nothing more — to keep the POC simple.

Message ID	Direction	Name	Purpose
0x0001	Device -> Server	Device General Response	Device confirms it received our message (success/failure/unsupported).
0x8001	Server -> Device	Server General Response	We confirm we received the device's message.
0x0002	Device -> Server	Heartbeat	Device says 'I'm still online.' Empty body.
0x0100	Device -> Server	Device Registration	Device introduces itself (only sent if authentication fails).
0x8100	Server -> Device	Registration Response	We confirm registration and issue an authentication code.
0x0102	Device -> Server	Device Authentication	Device sends its authentication code after reconnecting.
0x8900	Server -> Device	Downstream Passthrough	Wrapper used to send lock business commands (seal/unseal/etc.).
0x0900	Device -> Server	Upstream Passthrough	Wrapper used by the device to send lock business data/responses back.
0x0200	Device -> Server	Location Report	Device sends GPS location, speed, and alarm flags.
0x8001 / 0x0001	Both directions	Acknowledgement of Location Report	Standard general response to confirm receipt.

5.1 Lock Business Commands (carried inside the 0x8900 / 0x0900 wrapper)

These are the actual 'seal' and 'unseal' instructions. They travel wrapped inside message 0x8900 (server-to-device) and come back wrapped inside 0x0900 (device-to-server), each wrapped payload starting with the byte 0x81 to say 'this is lock business data.'

Command	Command Byte (Cmd)	Direction	Purpose
Seal (Lock)	0x32	Server -> Device	Instruct the lock to physically lock/seal.
Unseal (Unlock)	0x38	Server -> Device	Instruct the lock to physically unlock.
Clear Alarm	0x42	Server -> Device	Clear an active alarm/tamper condition on the lock.

Command	Command Byte (Cmd)	Direction	Purpose
Operation Reply	0x55	Device -> Server	The lock's reply to any of the above three commands, with a result code.
Lock Info Upload	0x01 (SubCmd)	Device -> Server	Lock proactively reports its own status.
Alarm Upload (cable cut / forced open / etc.)	0x02 / 0x03 / 0x04 / 0x05 (SubCmd)	Device -> Server	Lock proactively reports a tamper condition.

5.2 Result / Response Codes We Must Recognize

Code (hex)	Meaning	Dashboard should show
0x80	Seal successful	Green — 'Sealed'
0x81	Repeat seal (already sealed)	Yellow — 'Already sealed'
0x82	Lock not closed properly, seal failed	Red — 'Seal failed: lock not closed'
0x83	Battery too low to seal	Red — 'Seal failed: low battery'
0x84/0x85/0x86/0x87	Lock in an abnormal/tamper state, seal refused	Red — 'Seal failed: tamper condition present'
0x89	Seal command timed out	Red — 'Seal failed: no response'
0x90	Unseal successful	Green — 'Unsealed'
0x91	Repeat unseal (already unsealed)	Yellow — 'Already unsealed'
0x93	Wrong key/password, unseal refused	Red — 'Unseal failed: wrong key'
0x97	Attempted unseal on a lock that was never sealed	Yellow — 'Not sealed yet'
0x99	Unseal command timed out	Red — 'Unseal failed: no response'

5.3 Field Layout — Seal / Unseal Command (Server to Device)

This is the exact instruction our server builds and sends to seal or unseal the lock.

Field	Size	Description
Command Byte	1 byte	0x15 for lock operations (seal/unseal/clear alarm/set password)
Cmd	1 byte	0x32 = seal, 0x38 = unseal, 0x42 = clear alarm
LockID	4 bytes	The lock's unique ID (an 8-digit number converted to hex — see conversion rule in protocol Appendix 0)

Field	Size	Description
Gate	1 byte	Channel number; use 0x00 for the POC (single lock, no checkpoint/gate integration)
Bill	8 bytes	Cargo manifest/waybill number; for POC, can be a placeholder value
LineCode	2 bytes	Route code; placeholder value for POC
Key	6 bytes	The security key the lock checks before unsealing — must match what was set when sealed
ValidTime	1 byte	Only used for dynamic passwords; 0x00 for standard seal/unseal
Reserved	4 bytes	Unused, fill with zeros
Absolute Time	6 bytes (BCD)	Time the command was generated, e.g. 0x080402152050 = 2008-04-02 15:20:50

5.4 Field Layout — Operation Reply (Device to Server)

This is what the lock sends back after receiving a seal/unseal/clear-alarm command.

Field	Size	Description
Reply Byte	1 byte	Always 0x55 for an operation reply
Cmd	1 byte	Echoes which command this is a reply to
LockID	4 bytes	The lock's ID
Gate	1 byte	Channel number (echoed back)
Bill	8 bytes	Cargo manifest number (echoed back)
Voltage	1 byte	Battery voltage code — see Section 5.5
LockStatus	1 byte	Current lock status — see Appendix reference, Section 5.6
MotorStatus	1 byte	Motor status (raw value, display as-is for POC)
LineCode	2 bytes	Route code (echoed back)
Operation ID	8 bytes	Identifies who/what triggered this (see protocol doc — e.g., keyboard, remote, IC card)
Command Source	1 byte	0x00 SMS, 0x01 automatic, 0x02 keypad, 0x03 handheld device, 0x04 our platform, 0x05 checkpoint, 0x06 IC card
Absolute Time	6 bytes (BCD)	Time the lock generated this reply

5.5 Battery Voltage Code Reference

Hex Value	Voltage	Dashboard Label
0x30	3.3V	Critical — severe low battery
0x31	3.6V	Low battery
0x32 – 0x36	3.7V – 4.1V	Normal battery
0x37	4.2V	Fully charged

5.6 Lock Status Reference (top 4 bits of the LockStatus byte)

Value	Meaning	Dashboard Label
0x1x	Open state	Open
0x2x	Standby state	Standby
0x3x	Not closed properly	Warning — Not Closed
0x4x	Sealed (remote)	Sealed
0x5x	Sealed (locally, by keypad)	Sealed (Local)
0x6x	Unsealed (remote)	Unsealed
0xBx	Unsealed (locally, by keypad)	Unsealed (Local)
0x7x	Alarm state (cut/forced open/tamper/etc.)	ALARM
0x8x	Alarm state (local)	ALARM (Local)
0x9x	Alarm cleared	Alarm Cleared
0xAx	Abnormal / error state	Error

5.7 Field Layout — Location Report (Device to Server, Message 0x0200)

Field	Size	Description
Alarm Flags	4 bytes	Bitmask — bit 31 = illegal-open alarm, others cover speeding, fatigue, tamper, etc. (see protocol doc Table 24). For POC, decode at minimum: emergency alarm, tamper/illegal-open alarm.
Status Flags	4 bytes	Bitmask — includes ACC on/off, GPS fix valid/invalid, door open/closed. For POC, decode at minimum: GPS fix valid, and (if present) LockStatus value in bits 24–31.
Latitude	4 bytes	Degrees × 1,000,000 — divide by 1,000,000 to get real latitude.
Longitude	4 bytes	Degrees × 1,000,000 — divide by 1,000,000 to get real longitude.
Altitude	2 bytes	Metres above sea level.
Speed	2 bytes	Speed in units of 1/10 km/h — divide by 10 for km/h.

Field	Size	Description
Direction	2 bytes	Compass heading, 0–359 degrees, 0 = North.
Time	6 bytes (BCD)	Format YY-MM-DD-HH-MM-SS, already in India-adjacent GMT+8 in the source doc — treat as UTC offset to confirm with vendor for India deployment (see open item in Section 10).

6. Step-by-Step Message Flows (for POC Scope)

These are the exact conversations our server must be able to have with the lock device. Each flow below should become a test scenario.

6.1 Flow: Device Connects and Authenticates

4. Device opens a TCP connection to our server's IP and port.
5. Device sends message 0x0102 (Authentication) containing its authentication code (for a factory-default device, this is its SIM card ICCID plus device version — see protocol doc Section 8.8).
6. Server checks the code. If recognized, server replies with a General Response (0x8001) with result = success.
7. If the device is not recognized, it instead sends message 0x0100 (Registration). Server replies with 0x8100, assigning an authentication code the device will use next time.
8. From this point on, the connection is considered 'authenticated' and lock commands can be sent.

6.2 Flow: Heartbeat (Keep-Alive)

9. Every N seconds (device-configured), the device sends message 0x0002 (Heartbeat) with an empty body.
10. Server replies with General Response (0x8001), result = success.
11. If the server does not hear from the device for an extended period, the dashboard should mark the device as 'Offline.'

6.3 Flow: Operator Seals the Lock

12. Operator clicks 'Seal' on the dashboard.
13. Our API server builds a Seal command (Cmd = 0x32) with the correct LockID and Key, following the field layout in Section 5.3.
14. This command is wrapped inside message 0x8900 (Downstream Passthrough) and sent to the device over the open TCP connection.
15. Device physically seals, then replies with an Operation Reply (0x55) wrapped inside message 0x0900, including a result code (Section 5.2) and updated LockStatus (Section 5.6).
16. Server updates the database with the new status and the result, and the dashboard reflects the change (e.g., 'Sealed' shown in green, or an error shown in red).
17. This exact same flow applies to Unseal (Cmd = 0x38) and Clear Alarm (Cmd = 0x42).

6.4 Flow: Location & Alarm Reporting

18. Device periodically sends message 0x0200 (Location Report) with its GPS position, speed, and alarm flags.
19. Server replies with a General Response (0x8001).
20. Server decodes the alarm flags. If any tamper/emergency bit is set, an alert is raised on the dashboard immediately.

21. Location and any alarms are saved to the database as a new history entry (never overwritten, so we keep a full audit trail).

6.5 Flow: Simulator Mode (No Physical Lock Yet)

Before hardware arrives, we run a small script (the 'Device Simulator') that connects to our own server and plays out flows 6.1–6.4 using scripted, realistic values. This lets us build and demo the entire dashboard experience early. It should be built with configurable behavior so a developer/tester can trigger 'simulate a tamper alarm' or 'simulate a low battery' from the command line.

7. Database Schema (PostgreSQL)

A simple relational schema — five tables — is enough for the POC.

7.1 Table: devices

Column	Type	Notes
id	UUID, primary key	Internal record ID
lock_id	VARCHAR(8), unique	The 8-digit LockID from the protocol
terminal_id	VARCHAR(12)	The device's terminal ID (BCD-decoded)
auth_code	VARCHAR(64)	Authentication code issued at registration
current_key	VARCHAR(10)	The current seal/unseal security key
status	VARCHAR(20)	e.g., sealed, unsealed, alarm, offline
battery_voltage_code	VARCHAR(4)	Raw hex code from last reply, e.g., 0x35
last_seen_at	TIMESTAMP	Last message received from this device (used for online/offline)
created_at	TIMESTAMP	

7.2 Table: commands

Column	Type	Notes
id	UUID, primary key	
device_id	UUID, foreign key -> devices.id	
command_type	VARCHAR(20)	seal, unseal, clear_alarm
sent_at	TIMESTAMP	
result_code	VARCHAR(4)	Hex result code from Section 5.2, once replied
result_label	VARCHAR(100)	Human-readable label, e.g. 'Seal failed: low battery'
replied_at	TIMESTAMP	Null until device responds

7.3 Table: locations

Column	Type	Notes
id	UUID, primary key	

Column	Type	Notes
device_id	UUID, foreign key -> devices.id	
latitude	DECIMAL(9,6)	
longitude	DECIMAL(9,6)	
speed_kmh	DECIMAL(6,1)	
direction_deg	SMALLINT	
reported_at	TIMESTAMP	Device-reported time
received_at	TIMESTAMP	Server-received time

7.4 Table: alarms

Column	Type	Notes
id	UUID, primary key	
device_id	UUID, foreign key -> devices.id	
alarm_type	VARCHAR(50)	e.g., cable_cut, forced_open, low_battery, illegal_open
raised_at	TIMESTAMP	
cleared_at	TIMESTAMP	Null until cleared
raw_flags_hex	VARCHAR(16)	Original alarm-flags bitmask, kept for audit/debug

7.5 Table: message_log

Column	Type	Notes
id	UUID, primary key	
device_id	UUID, foreign key -> devices.id, nullable	
direction	VARCHAR(10)	inbound or outbound
message_id_hex	VARCHAR(6)	e.g., 0x0200
raw_bytes_hex	TEXT	Full raw message, hex-encoded — invaluable for debugging protocol issues
created_at	TIMESTAMP	
Why keep raw_bytes_hex:		

Column	Type	Notes
Since we're implementing a protocol from a written document without prior confirmed working code, keeping the exact raw bytes of every message is the single most useful debugging tool when something doesn't parse as expected — especially once real hardware is involved.		

8. API Endpoints & Dashboard Screens

8.1 REST API Endpoints (server exposes these to the web dashboard)

Method & Path	Purpose
GET /api/devices	List all known devices with current status.
GET /api/devices/{id}	Get full detail for one device, including latest location.
POST /api/devices/{id}/seal	Trigger the Seal command flow (Section 6.3).
POST /api/devices/{id}/unseal	Trigger the Unseal command flow.
POST /api/devices/{id}/clear-alarm	Trigger the Clear Alarm command flow.
GET /api/devices/{id}/locations	Location history for the map/track view.
GET /api/devices/{id}/events	Combined history feed: commands, alarms, location updates — newest first.
GET /api/devices/{id}/alarms	Alarm history, including active (uncleared) alarms.

8.2 Dashboard Screens

Screen	What it shows
Device List	All devices, each with an online/offline indicator and current lock status badge (Sealed / Unsealed / Alarm / Unknown).
Device Detail	Selected device's live status, battery level, a map showing last known location, and three buttons: Seal, Unseal, Clear Alarm.
Event Feed	A live-updating list on the Device Detail screen: every command sent, every reply received, every alarm raised — newest at top, with plain-language labels (from Section 5.2 and 5.6).
Location History	A simple line on the map showing recent movement, with timestamps.

For the POC, real-time updates on the dashboard can be done with simple polling (the browser asks the server for updates every few seconds) — no need for advanced real-time technology (e.g., WebSockets) unless the team wants a more polished demo.

9. Technology Stack & Build Order

9.1 Recommended Technology Stack

Layer	Technology	Why
TCP Listener / Protocol Parser	Node.js	Excellent raw binary/buffer handling; large ecosystem; same language can power the API layer.
Device Simulator	Node.js (small standalone script)	Reuses the same protocol-encoding code as the real server.
API Layer	Node.js + Express	Simple REST framework, quick to stand up.
Database	PostgreSQL	Reliable, well-supported, easy to query for audit history.
Frontend Dashboard	React (with a simple map library, e.g., Leaflet)	Component-based, fast to build simple dashboards; Leaflet is free and lightweight for showing a single point on a map.
Local Development	Docker Compose	Spins up server + simulator + database + frontend together with one command — makes the POC easy to demo on any laptop.

9.2 Suggested Project Folder Structure

```
indialock-connect-poc/  
  server/  
    src/  
      protocol/      <- byte encode/decode, escaping, checksum (Sections 4-5)  
      tcp-listener/  <- accepts device connections, hands off to protocol layer  
      services/      <- command service, location service, alarm service  
      api/           <- REST endpoints (Section 8.1)  
      db/            <- database models & migrations (Section 7)  
      tests/  
        protocol.test.js <- uses the worked example in Section 4.2 as a test case  
    simulator/  
      simulate-device.js <- pretends to be a lock device for early testing  
    dashboard/  
      src/  
        pages/       <- Device List, Device Detail (Section 8.2)  
        components/  <- status badges, map view, event feed  
    docker-compose.yml
```

9.3 Build Order (hand these to Claude Code one at a time, in order)

22. Build the Protocol module: byte-escaping, checksum, header encode/decode, and the Seal/Unseal/Operation Reply message encode/decode from Section 5. Write and pass the worked-example test from Section 4.2 first, before anything else.
23. Build the Device Simulator: a script that opens a TCP connection to a given host/port, sends a registration + auth handshake (Section 6.1), sends periodic heartbeats (Section 6.2), and

responds to Seal/Unseal/Clear-Alarm commands with a scripted Operation Reply (Section 6.3). Add command-line flags to simulate an alarm or low battery.

24. Build the TCP Listener + database layer: accept incoming device connections, persist devices/commands/locations/alarms/message_log per the schema in Section 7, and wire up the flows in Section 6.
25. Build the REST API layer (Section 8.1) on top of the database and command service.
26. Build the React dashboard (Section 8.2), pointing at the REST API.
27. Run the whole stack against the Simulator end-to-end and verify all 7 acceptance criteria from the Business Requirements Document, Section 8.
28. Once the physical lock sample arrives: update only the device's network configuration to point at our server (no platform code changes expected), and re-run the same acceptance criteria against real hardware.

10. Testing & Validation Plan

Test	Type	Pass Condition
Byte-escaping worked example (Section 4.2)	Automated unit test	Encoding 0x30 0x7E 0x08 0x7D 0x55 produces exactly 0x7E 0x30 0x7D 0x02 0x08 0x7D 0x01 0x55 0x7E.
Checksum calculation	Automated unit test	XOR of a known sample message matches a manually calculated value.
Registration & authentication handshake	Integration test (simulator)	Simulator completes flow 6.1 and server marks device as authenticated.
Heartbeat / online-offline detection	Integration test (simulator)	Server marks device offline if no message received within the configured timeout.
Seal / Unseal / Clear Alarm round-trip	Integration test (simulator, then hardware)	Dashboard reflects correct status after each command, matching Section 5.2 result codes.
Location report decoding	Integration test (simulator, then hardware)	Latitude/longitude/speed shown on dashboard match the values sent (after unit conversion).
Alarm detection & display	Integration test (simulator, then hardware)	Triggering a simulated tamper alarm shows up on the dashboard within a few seconds.
Full hardware validation	Manual, end-to-end	All items in the Business Requirements Document's Section 8 (Acceptance Criteria) pass using the real physical lock sample.

11. Known Gaps & Assumptions for the POC (Track These)

These are places where the protocol document was incomplete, ambiguous, or where we made a deliberate simplification for the POC. Keep this list updated as development proceeds — it becomes the punch-list for what needs vendor confirmation before going to production.

#	Gap / Assumption	POC Approach
1	Protocol doc doesn't specify TLS/encryption as mandatory.	Run unencrypted for POC; restrict access via firewall/IP allow-list.
2	GBK text encoding for STRING fields may not be needed if we never send Chinese text.	Treat all STRING fields as plain ASCII/UTF-8 for POC; flag for review if vendor confirms GBK is required on the wire.
3	Document distinguishes '7B lock' vs '4G lock' hardware variants with slightly different fields (e.g., cell-tower info size).	Confirm which variant the physical sample is once delivered; build the location parser to handle the variant actually received.
4	Absolute Time fields are described as GMT+8 in the source document (written for a China deployment).	Store all timestamps as UTC internally; confirm with vendor whether the physical device's clock can be set to a different timezone or if we should always assume GMT+8 and convert.
5	LockID conversion rule (8-digit decimal number to 4-byte hex) is documented but untested against real hardware.	Implement exactly as documented (Appendix 0 of protocol doc); validate against the real LockID printed on the physical unit once it arrives.
6	No field in the protocol doc directly maps to 'authorized user' — operation source codes exist but no user-identity model.	For POC, assume a single internal operator; do not build multi-user permissions.

12. What We Will Need From the Product Team for a Full-Scale Live Product

This POC is intentionally narrow. Below is the list of decisions, resources, and confirmations the Product team (working with the vendor, legal, and infra teams) will need to provide before this can move from 'proof of concept' to a real, live, commercial product.

12.1 Hardware & Vendor Relationship

- A committed supply agreement with the lock vendor/OEM for production volumes, including lead times and pricing.
- Direct engineering contact with the vendor for protocol clarifications and firmware issue escalation — not just documentation.
- Confirmation of which hardware variant(s) will be deployed in India (the POC will validate whichever sample was sent; production may involve other variants mentioned in the protocol document, e.g., 4G LBS models).
- A vendor roadmap / support commitment for firmware updates and bug fixes over the product's lifetime.

12.2 Regulatory & Compliance (India-specific)

- WPC/ETA type-approval status of the cellular module used in the lock, for legal operation in India.
- Confirmation of SIM/IoT registration requirements with Indian telecom operators (e.g., bulk IoT SIM provisioning with Airtel, Jio, or Vodafone Idea).
- Legal review of data localization requirements if location/cargo data is considered sensitive under Indian data protection law.
- If used for regulated cargo (e.g., customs-sealed containers), confirmation of any government/customs integration requirements (e.g., linkage to e-Way Bill or customs sealing standards).

12.3 Security

- A decision and budget for a proper security layer (e.g., TLS/VPN tunnel for the device connection) before any live cargo deployment — the protocol itself does not mandate encryption.
- A key-management policy for the lock/unlock security keys (rotation, storage, who can issue them) — the protocol has the mechanism but no policy is defined.
- A security review / penetration test before go-live, since we are exposing a public TCP port to the internet for device connections.

12.4 Product & Business Requirements

- Multi-tenant design requirements: will this serve one internal fleet, or multiple customer companies with separate logins and data isolation?

- User roles and permissions model (e.g., who can seal/unseal, who can only view).
- Integration requirements with existing systems (e.g., a Transport Management System, ERP, or customer-facing tracking portal).
- Reporting and analytics requirements (e.g., dashboards for fleet managers, compliance reports for auditors).
- Mobile app requirement, if field staff need to trigger seal/unseal from a phone rather than a web dashboard.

12.5 Engineering & Operations Scale-Up

- Expected device fleet size (tens vs. thousands of locks) — this drives infrastructure sizing, database design, and whether we need a message queue between the TCP listener and the rest of the system.
- High-availability and failover requirements (a lock in transit cannot afford a platform outage during a seal/unseal attempt).
- Monitoring, alerting, and on-call support model for production incidents.
- SLA commitments to end customers (e.g., alarm-to-notification latency guarantees).
- Budget and timeline for a dedicated QA/hardware test lab, since ongoing hardware variance testing will be a recurring need, not a one-time POC activity.

Summary for Product Leadership:

The POC will prove the software concept works. It will not, by itself, answer the regulatory, security, vendor-commercial, or scale questions above — those require Product, Legal, and Infrastructure involvement in parallel with (or immediately after) the POC, before any commitment to a live customer rollout.